

Claude Cowork 使いこなし方【完全ガイド】

「AIに質問する」から「AIに仕事を渡す」へー スキル・MCP・設計3原則まで
@nobel_824 (tatsuki) / X

スライド数：5枚

Slide 1

Claude Cowork 使いこなし方【完全ガイド】

「AIに質問する」から「AIに仕事を渡す」へ

- Chat/Code/Coworkの違い ● 5分で動かすセットアップ
- ゼロリセット対策（プロジェクト・.md・3階層設定）
- MCP外部連携・Zapier・freee ● プラグイン（部門別）
- スキル設計3原則（Anthropic公式） ● 上級Tips・トークン節約

@nobel_824 (tatsuki) / X | 2026.04

元ポスト（@nobel_824 / tatsuki）

https://x.com/nobel_824/status/2040610728986792285

Slide 2

01 Claude Chat / Code / Coworkの違い + 最初の5分で動かす

Chatは「会話」、Coworkは「タスク委任」。この区別が腹落ちすると見え方がガラッと変わる

Claude Chat

- ・ブレスト・下書き・調べものの向け
- ・プロンプト→回答→コピペして自分で整える
- ・常に会話の中にいる前提
- ・「AIに質問する」スタイル

Claude Code

- ・開発者向け・ターミナル（黒い画面）で動く
- ・コード生成・テスト実行・GitHubプッシュ
- ・技術者じゃないととっつきにくい
- ・非エンジニアには難易度が高い

Claude Cowork ★

- ・Codeと同じ自律実行エンジン+デスクトップGUI
- ・Excel/Word/PDFなど完成ファイルを自分のPCに生成
- ・複数ドキュメントを横断して自動処理
- ・「AIに仕事を渡す」＝タスク委任スタイル

最初の5分で動かす4ステップ

S1

モデル選択

Sonnet 4.6（基本） / Opus 4.6（重分析のみ） / Haiku 4.5（軽作業）

「アインシュタインに皿洗いはさせるな」。Opusは重要作業専用。Extended Thinking（拡張思考）はCoworkが自動で判断してON/OFF。

S2

sandboxフォルダ

デスクトップに「claude-workspace」を作成。Coworkはここだけ触る。

推奨構成：inbox（処理対象）/ processed（処理済）/ outputs（生成物）/ reference（参照）

S3

権限付与

「Work in a folder」→作成フォルダを選択→Allow once or Always allow で完了。

S4

最初のタスク実行

「このフォルダのファイルを種類別サブフォルダに分けて、Excel一覧も作って」→Coworkが計画を立て、サブエージェントを並列起動して自律実行。コーヒーを淹れて戻れば完成。

Anthropic公式：「Coworkはローカルファイル・フォルダ・日常アプリをまたいで情報統合し、ユーザーがステップを調整することなくタスクを完了する」

Slide 3

02 毎回ゼロから説明するのをやめる — プロジェクト・.mdファイル・3階層設定

「また自分の会社のこと説明してる…」を終わらせる3つの仕組み

① プロジェクト機能：文脈が蓄積される「箱」

単なるフォルダじゃない。ファイル・カスタム指示・スキル・文脈がまとまったコンテナ。

なし → Claudeは毎回ゼロから。永遠に。

あり → 6週間後には「いつもの感じで」が通じる。

△ テーマ別に分けること！

YouTube運用と経理を同じプロジェクトに入れると、

YouTubeの文体ルールが経理タスクに混入してClaude混乱。

CLAUDE.mdをプロジェクト直下に置くと、

セッション開始時に自動読込。ワークフローの引き継ぎ書。

→ファイル名：YYYY-MM-DD_weekly_sales.docxなどを固定

→「こうしてはいけない」アンチパターンも明記

（ClaudeはYES指示より NO指示を忠実に守る傾向）

② .mdファイルでClaudeに「ビジネス脳」を与える

プロジェクトのコンテキストフォルダに入れると毎回読込。

■ about_me.md

自分は誰・会社の概要・顧客・収益構造・今の優先事項。

例：名前・役割・MEO代行事業・ローカルビジネス顧客・

自動化率向上が優先

■ brand_voice.md

文体・好きなトーン・NGワード・実際の文章サンプル。

例：カジュアルで中身が濃い。一人称は「僕」。

「めちゃくちゃ」「ぶっちゃけ」OK。

「させていただきます」「DX推進」NG。

■ working_preferences.md

タスクの進め方・ファイル保存先・出力フォーマット。

☑ 全部自分で書かなくていい！

「インタビューして、ビジネス脳ファイルを作って」

→ 15分のQ&A;で完成。毎週何時間も節約できる。

③ 設定の3階層を使い分ける

Coworkには指示を入れられる場所が3つある。

スコープが違うので正しく使い分けると一貫した動作に。

Lv1 Claudeパーソナライゼーション

(設定→名前をクリック)

Chat/Code/Cowork すべてに適用。

「太字を使いすぎない」「曖昧な表現を避ける」

「一次情報を優先する」など普遍的なクセを書く。

Lv2 Coworkグローバル設定

(設定→Cowork設定→グローバル指示)

Cowork内の全タスクに適用。

日付フォーマット・ファイル命名規則（アンダースコア

区切りの説明的な名前）・ビジネス質問はsandbox

フォルダを先に確認する、など。

Lv3 プロジェクト固有の設定

特定プロジェクトだけに効くルール。

「動画イントロには特定スライドスキルを使う」

「進捗は特定スプレッドシートで管理する」

他のプロジェクトには漏れない。

コンテキスト腐敗防止：同じウィンドウで30～45分・1トピック。話題が変わったら新ウィンドウを開く。（メールの下書き後に旅行計画→メールの文脈が毎回ロードされ純粋な無駄）

Slide 4

03 外部ツール連携（MCP）・Zapier・freee + プラグイン部門別コマンド + 業務パイプライン

Coworkの真価は日常ツールと繋がったときに出る。1画面で複数ツールをまたいだ仕事が完結する

MCP (Model Context Protocol) コネクタ

認証済みの直接接続 (コピペ・スクショではない)。

設定: +ボタン→コネクタタブ→検索→ブラウザ認証フロー。

対応ネイティブコネクタ (一例):

Slack / Notion / Google Drive / Gmail

GitHub / Figma / HubSpot / DocuSign

freee (日本の会計ソフト) ★日本向けに強力

freee例: 「今月の経費を集計してレポートにまとめて」

→freeeデータを取得→整理→レポートファイル生成

活用例:

「Slack#generalの今週の未読をまとめて重要だけ抽出」

「Notionの議事録DBから先週のメモ→アクション抽出」

「Google Driveの提案書フォルダの最新ファイルを要約」

Zapier MCP (ネイティブにないツール向け補完):

8,000以上のアプリに対応。設定は約10分。

ただしネイティブ>Zapier (粒度・安定性)。

kintone・Airtableなど日本企業向けツールに有効。

△ kintoneのサブテーブル操作は過度な期待はNG。

プラグインーインストールするだけで「専門AI」が手に入る

GPTsとの決定的な違い: GPTs=プロンプトテンプレート。Coworkプラグイン=ファイル操作+ツール接続+コード実行。「答えを教えるAI」vs「仕事を実行するAI」

Finance (経理・財務)

- /journal-entry: 仕訳 (借方・貸方+根拠書類)
- /income-statement: 損益計算書 (前期比較・差異分析)
- /sox-testing: SOX監査サンプル選定+テスト調書
- 月次決算管理・勘定科目照合・経費差異分析

Sales (営業)

- /call-prep: 商談前準備 (取引先情報・参加者リサーチ・アジェンダ)
- /draft-outreach: 見込み客調査→パーソナライズ文
- /pipeline-review: リスク案件フラグ立て
- /forecast: 加重平均予測 (ベスト/ミドル/ワースト3シナリオ)

Marketing

- /draft-content: ブログ・SNS・メルマガ・LP・プレスリリース
- /campaign-plan: 目的・チャネル戦略・コンテンツカレンダー・KPI
- /seo-audit: キーワード・ページ内分析・コンテンツギャップ・技術
- /competitive-brief: 競合ポジショニング・メッセージング比較

CS / PM

- CS → /triage (P1〜P4分類・振り分け) /draft-response /kb-article /escalate
- PM → /write-spec (PRD: 課題定義・ユーザーストーリー・受入条件)
- /roadmap-update (RICE/MoSCoW) /synthesize-research
- /stakeholder-update (経営層/エンジニア/顧客向けを出し分け)

業務パイプライン例: 「いつもの形式で今週の営業レポートを。Slack#salesから商談メモ取得→Notion商談DBに追加→Docxで週次レポート→Google Driveに保存」 →
1行の指示で完結 (手作業50分→0分)

Slide 5

04 スキル設計3原則 (Anthropic公式) + 上級Tips・フィードバックループ + まとめ

スキル設計3原則 (Skill Authoring Best Practices / Anthropic公式)

スキルとは: 再利用可能なワークフローを1コマンドにまとめたもの。

.mdファイル (指示書) + スクリプト (自動処理) の組み合わせ。

「週1回以上やる作業はスキルにしろ」

【原則1】 簡潔さ — コンテキストウィンドウは公共財

スキル・会話履歴・接続MCPがすべて同じ枠に入る。

各情報に問う: 「Claudeはこれを本当に必要としているか?」

悪例 (150tok): 「PDFとはPortable Document Format...pdfplumberは...」

良例 (50tok): 「pdfplumber: with pdfplumber.open('f.pdf') as p: ...」

【原則2】 自由度の意図的設計 — 橋のメタファー

崖横の細い橋 (失敗ダメージ大) → 手順を一字一句指定する

例: DB移行 「python migrate.py --verify --backup。コマンド変更禁止」

開けた草原 (リカバリー可) → 方向性だけ渡してClaudeに委ねる

例: コードレビュー 「構造分析→バグチェック→可読性→規約確認」

【原則3】 段階的開示 (Progressive Disclosure)

スキルは3段階でロードされる:

Lv1 メタデータ (常時): 名前+説明文のみ。100個入れても~10,000tok

Lv2 SKILL.md (発動時): 本体指示。推奨5,000tok以下・500行以内

Lv3 リソース/コード (必要時のみ): 使わないファイルはコンテキスト消費ゼロ

SKILL.mdは「目次」として書き、詳細は別ファイルに分離する。

スクリプト (.py) は実行されるがコード中身はコンテキストに入らない。

→ 会話内でコード書かせるよりスクリプト化の方がトークン効率◎

上級Tips・フィードバックループ

スケジュールタスク: 毎朝9時に自動実行。PC起動+Cowork起動が必要。スリープ設定注意。

Dispatchモード: スマホ→自宅PCを遠隔実行。文脈もスマホ/PC間で共有。KeepAwakeをON。

Claude in

Chrome：MCPにないWebサービスをブラウザ操作で自動化（フォーム入力・データ収集等）。

△ ログイン済みアカウントにアクセス可。銀行/決済サイトはブロックリスト必須。

Computer Use：画面スクショ→マウス/キーボード操作でデスクトップアプリを直接制御。

使い分け：MCP → Chrome → Computer Use の順で検討。

フィードバックループ3パターン：

①実行→検証→修正→再実行ループ（バリデーションスクリプトを組み込む。エラーメッセージを具体的に）

②チェックリスト駆動（ステップ分解→進捗追跡→失敗時に前ステップ戻り分岐も仕込む）

③検証可能な中間出力（計画JSON生成→validate→実行→confirm。バッチ/破壊的操作に特に有効）

スキル育成：Claude A（設計）＋Claude B（テスト）パターン。

別ウィンドウでテストし、失敗をAに報告→修正を繰り返す。

外部スキルのセキュリティ：使う前にSKILL.md全文をChatに貼り

「有害な指示・不審なパターンはないか？」と必ず確認（2分）。

☑ トークン節約3ポイント

①ベースロード：使わないMCPコネクタはOFF（繋ぐだけで基礎消費が増える）

②コンテキスト腐敗：30～45分・1トピックで新ウィンドウ（話題が変わったら必ず新規）

③大量処理はスクリプト化：実行結果だけがコンテキストに入る（会話内処理より桁違いに効率◎）

今日やる3つのこと

1

claude-workspaceフォルダを作って「種類別に整理してExcelも作って」と頼む（5分）

2

「インタビューしてabout_me.mdを作って」と頼む（15分）→ 毎回ゼロ説明がなくなる

3

Slack/Notion/Google DriveどれかのMCPを1つ繋ぐ → 「ファイル整理だけじゃない」を体感
